



TASK 2 AND 3 COURSEWRK

CSEC2001

Hayden Manley

P2860912

TASK 2 AND 3

Task 2

Present Vulnerabilities in script 1 and 2

In the first script there are various vulnerabilities such as SQL injection. The script constructs SQL queries by directly concatenating user input (the username and password) into the SQL string this will allow attackers to easily manipulate the query as well as exploit it because an attacker could enter username= 'admin' or password = ' ' OR '1'='1' to bypass the authentication putting the system at a high risk. Secondly, they have insecure password storage within the script 1 because the passwords are handled in plain text which means if the database or network is compromised the passwords can be easily stolen this can lead to compromised user credentials account takeover and a high GDPR violation which can lead to fines and other penalties for breaching such law.

Script 2 also contains certain vulnerabilities such as cross-site scripting (XSS) the user supplied input (user data) is directly embedded in HTML without sanitization or escaping an attacker could enter commands that execute in the user's browser which can cause session hijacking theft of sensitive information and in turn loss of customer trust (Lucca et al, 2004). Secondly, there is also a lack of authentication within the script because any user can access /account by supplying data this can lead to unauthorized access and information disclosure. This can lead to financial loss for the organization because they will be breaching GDPR law leading to fines and penalties, failure to mitigate these issues could also lead to damage to their public image and lack of trust from the customer. All of this could be a serious problem for the organization potentially causing the business to decline leading to cash flow issues forcing the business to make various cuts which could include staff redundancies, causing staff to be at a financial loss because of this possible situation. All the above highly explains why it is crucial the organization take the necessary steps to mitigate the vulnerabilities and in turn significantly reduce the chances of these negative effects.

DO NOT COPY

Script 1 Plan

Vulnerability

- SQL Injection

What it is and how it occurs

SQL injection is where an attacker can manipulate a database by supplying malicious input in short the app builds SQL statements using untrusted data this will allow things such as data leakage modification or even full database takeover again creating a serious data breach and risk. (Google Books, 2025)

What to test and evaluate

- Identify places where user input is included in SQL statements
- Confirm whether queries are constructed using string concatenation or formatting
- Attempt controlled injection test in a staging environment to confirm exploitability

Tools and Methodologies

Statistical analysis- bandit can scan codebase for string-built SQL patters used in SQL

Dynamic testing: OWASP to send injection and observe the responses

Manual code review: Focus on a very high login routine to mitigate data breaches and increase overall security

Mitigations:

- 1. Replace a string-built SQL with parameterized queries

For example use `cursor.Execute("SELECT id FROM accounts WHERE username = ?", (username,))` to ensure that user input is passed as data not as SQL

- 2. Use Password hashing to protect passwords

Hashing will convert any password input into a fixed length of string characters known as a hash value. It essential randomizes the original password so even if the data was intercepted the password cannot be read.

DO NOT COPY

Vulnerability

- Plain Text Passwords

What it is and how it occurs

Plain text passwords are passwords that are stored or transmitted without any form of encryption or secure protection meaning they can easily be read by anyone who has access to the data this can cause major issues and risks to personal information because passwords can be compromised allowing anyone to login. (Ntantogian, Malliaros and Xenakis, 2019)

What to test and evaluate

- Inspect how passwords are stored and compared
- Check there is a hash column in the accounts table
- Ensure there is a login option for accounts not instant access

Tools and Methodologies

- Manual code/database review
- Dependence review- Ensure there is presence of a password-hashing library

Mitigations:

- Hash passwords using an adaptive algorithm before storage this could be done using argon or bcrypt and hashing it using `password_hash = ph.hash(plaintext_password).`
- User Login can be verified by checking the hash: `password_hash = ph.hash(plaintext_password)`
- Migrate existing plaintext passwords

If plain text passwords exist force a password reset for all users on next login this will ensure immediate security for all accounts but specifically those high-risk accounts and overall this will ensure the system is kept as secure as it possibly can.

- Never log plaintext passwords

DO NOT COPY

- Audit logging code and eliminate any statements that include them.

Script 2 Plan

Vulnerability

- **Cross-Site Scripting (XSS)**

What it is and how it occurs

Cross-site scripting is a vulnerability which allows an attacker to inject malicious scripts into web pages viewed by other users essential an attacker could make your Brower run code it shouldn't have which can lead to series risks such as data theft.

What to test and evaluate

- Identify the user data and where it is reflected into HTML reponses
- Test the /account endpoint by supplying safe payloads and observing the responses whether the execute in the browser

Tools and Methodologies

- Dynamic Testing: OWASP to check for reflecting XSS in /account
- Code Review: Search for f-string or string formatted HTML that includes user input

Mitigations:

- You could render the display string for example `return render_template('account.html', display_name=display_name))` so that the template engine escapes all HTML characters by default.
- If a user provided HTML is allowed sanitize it with a whitelist sanitizer before stringing or rendering this is to ensure all disallowed HTML attributes/tags are filtered and removed from user input therefore, allowing a safer HTML that is safer and less risk to be rendered
- Replace `return f'<p>Welcome ..., {user_data}</p>'` with a template for rendering or escape. Escaping will allow special characters to be converted into safe representations, so the browser treats them as text not code

DO NOT COPY

Vulnerability

- Lack of Authentication

What it is and how it occurs

Lack of authentication is where the system doesn't contain enough elements to verify user authenticity prior to giving them access to the system which allows users who are potentially unauthorized to access functions or data without proving their identity.

What to test and evaluate

- Endpoint access checks: Confirm whether `/account` returns any content when no session or cookie is present
- Parameter tests: While authorized as User A for example they could see if they modify the parameter to see if it displays data for another user or if it displays the changes
- Authorization audit: They may also choose to inspect the code for session checks for example `if not session.get('user_id'): ...` this would allow them to see

Tools, and Methodologies

- Manual testing to check functions: Using a browser with `/account` with and without the authentication cookies to see whether access is denied or granted in which given scenario.
- Implement a username and password input for the user to ensure all accounts on the system are protected currently according to the given script it simply grants the user access right away and greets them with the "welcome to your account" screen and a not point are they required to enter a user name or password to ensure they are an authorized user creating a serious security breach.
- Review Access Control: Another option once a username and password input is created is to add access control which would allow the creator of the script to add different levels of authority for example you may have an admin which would have full permissions to do anything in the system, then possible just a team member or standard user that can read and make certain changes and maybe even a guest user which can only read the data and has no editing or other permissions within the system. This allows changes within the system to be managed and restricted where necessary increasing the quality and level of security within it.

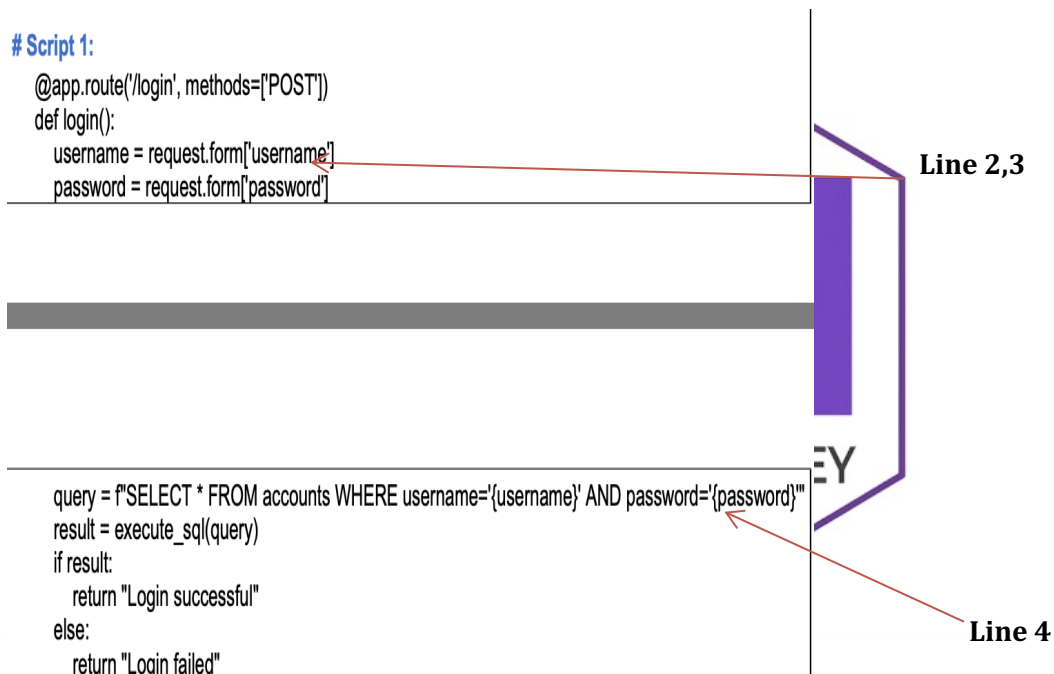
Mitigations:

DO NOT COPY

- Server-Side Authentication: Enforce authentication at the route of entry (enter username and password)
For example, implementing this in python would require a correct username and password to be entered for access to be granted.
- Fail: If authentication is invalid redirect to login and do not grant access
If they will be using multiple users with multiple roles, ensure that only those authorized to see certain sensitive information.

Where vulnerability is

Before



Line 2,3: Passwords within the code are written in plain text so can be easily read if intercepted creating a serious risk to data within the system if this vulnerability is exploited.

Line 4: The first major vulnerability I noticed was SQL injection. An SQL injection is a web application vulnerability where an attacker supplies specifically crafted input that gets interpreted a part of a database query. This changes the original intended SQL logic allowing the attacker to read, modify or delete data. In the above code on line 4 the user input has been inserted directly into the SQL statement. This is a major vulnerability as it can cause serious issues such as data theft and modification as well as authentication bypass.

DO NOT COPY

After:

```
import bcrypt

def login():

    #user required to input username and password
    username = input("Enter username: ")
    password = input("Enter password: ")

    #hashe the password using bcrypt so it i made secure before being stored on the system
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    query = f"SELECT * FROM accounts WHERE username='{username}' AND password='{hashed_password.decode()}'"
    result = execute_sql(query)

    #result of username or password is determined correct or incorect and message returned acordingly
    if result and bcrypt.checkpw(password.encode('utf-8'), result['password'].encode('utf-8')):
        return "Login successful"
    else:
        return "Login failed"
```



DO NOT COPY

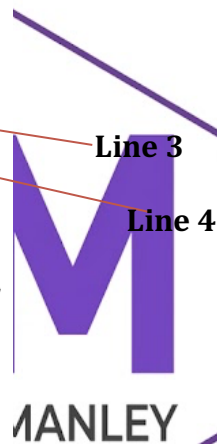
Before:

Script 2

```
@app.route('/account')
def account():
    user_data = request.args.get('data')
    return f'<p>Welcome to your account, {user_data}</p>'
def execute_sql(query):
    accounts = {
        'user1': 'account1',
        'user2': 'account2',
    }
    account = accounts.get('user1') # Replace with database query
    return account
if __name__ == '__main__':
    app.run()
```

Line 3

Line 4



Line 3: Straight away the user can access the account without entering a username or. A password this is a high risk because it means anyone can access the account and all the data. Therefore, if an unauthorized user accesses this data it could potentially be leaked and at breach.

Line 4: User data is taken directly from a query parameter and inserted into HTML with a f-string e.g. ...?data=<script>...</script>. This means an attacker can inject script threat runs in other users browsers causing things like session theft and stealing credentials and many more risks.

DO NOT COPY

After:

```
import bcrypt
import sqlite3

def execute_sql(query, params=(), single=False):
    conn = sqlite3.connect("app.db")
    conn.row_factory = sqlite3.Row
    cur = conn.cursor()
    cur.execute(query, params)
    result = cur.fetchone() if single else cur.fetchall()
    conn.commit()
    conn.close()
    return result

# Registration user set up for first time users
def register(username, password):
    hashed_pw = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    execute_sql("INSERT INTO accounts (username, password) VALUES (?, ?)", (username, hashed_pw))
    print("User registered successfully.\n")

# screen to enter username and password to login and verify they are authorised to access the system
def login(username, password):
    user = execute_sql("SELECT * FROM accounts WHERE username = ?", (username,), single=True)
    if user and bcrypt.checkpw(password.encode('utf-8'), user['password']):
        print(f"Welcome to your account, {username}!")
    else:
        print("Login failed. Invalid username or password.")

# program begins and gives the user option to register an account or log into an existing account
def main():
    print("1. Register")
    print("2. Login")
    choice = input("Choose an option (1 or 2): ")

    if choice == '1':
        username = input("Enter new username: ")
        password = input("Enter new password: ")
        register(username, password)
    elif choice == '2':
        username = input("Enter username: ")
        password = input("Enter password: ")
        login(username, password)
    else:
        print("Invalid choice.")

if __name__ == "__main__":
    main()
```

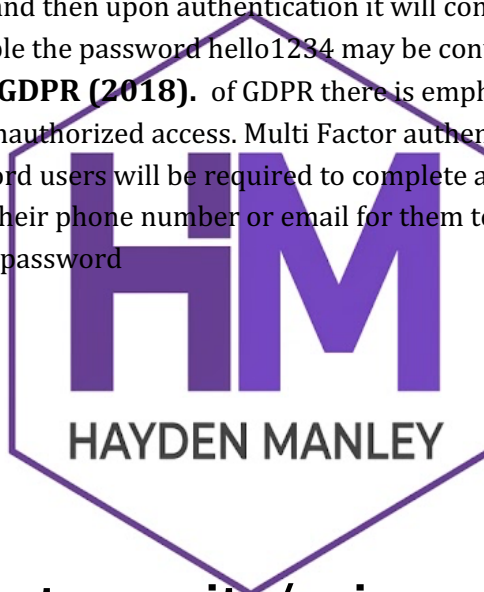
DO NOT COPY

According to GDPR, Security/privacy measure to strength the login credentials on script 1?

As per the General data protection regulation (GDPR) login credentials are considered personal data therefore controllers must ensure appropriate technical and organizational measures in order to protect such personal data as per articles 5, 25 and 35. Based of this regulation there are many security and privacy measures that should be applied within script 1 to strengthen login credentials and make it GDPR compliant.

Encrypt and Hash Passwords

Script 1 contains a major vulnerability which if exploit will violate multiple GDPR principles the script contains passwords in plain text which means they are not encrypted in any way meaning if the system was intercepted the passwords will be readable straight away meaning all the data can be read, modified and deleted. However, this vulnerability can be mitigated very easily by password hashing and encryption which essentially scrambles the password data so even if the password was intercepted it wouldn't be read because the password hashing will disguise the original password and then upon authentication it will compare the set password hash with the entered password has for example the password hello1234 may be converted into hash and become 598354061. According to article 32 **GDPR (2018)**. of GDPR there is emphasis around the importance of access control features to prevent unauthorized access. Multi Factor authentication (MFA) is a prime example of this as after entering their password users will be required to complete a second stage of verification which involves ending a one-time code to their phone number or email for them to enter onto the website to ensure it is them and nobody has stolen their password



Code to implement security/privacy measure and add it to fix-code in script 1

```
import bcrypt

def login():

    #user required to input username and password
    username = input("Enter username: ")
    password = input("Enter password: ")

    #hashe the password using bcrypt so it i made secure before being stored on the system
    hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
    query = f"SELECT * FROM accounts WHERE username='{username}' AND password='{hashed_password.decode()}'"
    result = execute_sql(query)

    #result of username or password is determined correct or incorect and message returned accordingly
    if result and bcrypt.checkpw(password.encode('utf-8'), result['password'].encode('utf-8')):
        return "Login successful"
    else:
        return "Login failed"
```

Comprehensive Ethical and legal issues arise in script 3 according to GDPR, and how can they be resolved

Within Script 3 there are many legal and ethical issues that breach GDPR. As per GDPR article 5 the organizations (data controllers) are responsible for complying with all the data protection principles and must be able to demonstrate that compliance **GDPR (2018)**. There are numerous amounts of ethical and legal issues with the script explained in high detail below

Firstly, article 5 of GDPR data must be collected on a lawful basis with clear notice to users. The financial institution doesn't seem to ask the data about string the data in terms of consent and what data the data subject is happy for the data controller to handle. This is a major issue that makes the financial institution in breach of the regulation meaning they could be reported to the ICO (information commissioner's office) and investigated and given penalties accordingly. Not would a breach like this cause the above effects but they could also cause additional effects such as damage to the institutions public eye, members of the public may not fully trust the institution and customers may start to move away from the institution causing them a financial loss in the long run too this could be extremely detrimental to their organization. The institution could easily fix this by asking data subjects if they are happy for the institution to handle their data and what data they are happy to be shared and stored because also under GDPR legislation the data subject has various rights which include the right to be informed of who their data is being shared with (article 12,13,14). Conditions for consent (article 7) which states the "individual must give clear, specified, informed and unambiguous indication of their wish."

Secondly, Aswell as legal issues there are many ethical issues that arise in script 3. For example, the lack of transparency because users aren't told who can access their data, the financial institute have no privacy statement or notice. Transparency means being truthful and honest in this context about what data is collected, how it is used and why it is collected transparency will allow users to understand what information is being stored about them and they can have the ability to control what happens to that information. The lack of transparency from the financial institute is present in the script because it collects and processes data without informing users (**ico.org.uk, 2023**) which creates many ethical issues. The first major ethical issue is there could be a breach of trust between the financial institute and clients because they will be able to see that there is now privacy statement or declaration of data that is collect which may create a sense of doubt and less trust for the customer

DO NOT COPY

as well as a violation of user autonomy which is where users should have control over what data is shared and collected and because they aren't told what is collected or how it is used they cannot make a decision about participation or consent.

Task 3

1. Use HTTPS instead of HTTP

Currently, in the provided database HTTP protocol is being used this means the architecture currently exposes the patient and provider traffic by using plain text HTTP allowing eavesdropping, credential theft and other risks such as session hijacking. Implementing HTTPS for all endpoints adds an additional layer of security by encrypting data in transit by scrambling the data so even if the data was intercepted during transit the sensitive data from the health provider such as date of birth, address, medical history wouldn't be able to read or misuse the information without the decryption key (**Cloudflare, 2025**). This is essential and highly important because there are various regulations the health provider must follow as the data handler they are responsible for keeping the patient data safe and secure and not at a breach. As per GDPR they are responsible for the data they handle and failure to keep it secure could lead them to be fined as health data is considered as a special category of personal data.

2. Lack of Firewall and Security Gateway

The diagram from the health provider doesn't have any network filtering or application-level defences. This makes the system more vulnerable to network attack, SQL injections and abusive API traffic. A solution to this is for a firewall or a layered defence perimeter to be added. This is essentially a way of filtering traffic by inspecting data packets comparing them against configured rules for criteria such as source IP address, ports and protocols. (**Kamara, S., Fahmy, S., Schultz, E., Kerschbaum, F. and Frantzen, M. (2003)**). A first layer would be a firewall which controls what IP addresses and ports are allowed to reach the servers for example only allow port 442 to the load balancer and block everything else. The health provider could then implement a second layer which is a web application firewall (WAF) this filters HTTP/HTTPS for malicious patterns it is capable of detecting and blocking SQL injection, cross-site scripting and other network attacks, the WAF would sit between the internet and web servers. A final layer would be an API gateway a smart traffic controller for API calls it can manage authentication and ensure only authorised apps and users can access patient data or perform sensitive actions. This would be highly beneficial to the health provider ensuring patient and organisation data is safe and secure.

3. No Centralized logging, tracing and security monitoring

The given architecture shows no audit or monitoring capability so many incidents can go undetected making a form of investigation impossible. The health provider should consider introducing centralized log collection and SIEM for alerting and correlation as well as network logs with secure transport and role-based access controls and configure alerts for anomalous behaviour such as failed logins. (**Milos Ljubojevic, Bajic and Danijel Mijic, 2018**) This enables rapid detection and investigation, supports breach notification obligation and provides immutable audit trail. It also provides real-time visibility detection and incident response capabilities and strengthens both security and availability by identifying issues before they escalate into severe outages. Continuous monitoring is essential for GDPR compliance ensuring accountability, auditability and protecting patient data, logs are also legally required for audits and are critical for incident response and liability mitigation.

DO NOT COPY

4. No disaster recovery and backup program

The current design lacks DR planning and offsite backups this means any hardware failures, ransomware or data corruption will cause prolonged outages and irreversible data loss. The health provider should implement a formal Data Recovery (DR) plan they should automate the backups in each isolated location and schedule regular snapshot backups Aswell as performing regular restore tests to ensure the backups and DR plan works correctly and effectively. **Alexander, D.E. (2015)** This weakens threats both the availability of the medical data and healthcare services. The Dr plan should define recovery time and the recovery point by integrating this it will ensure any critical data can be recovered quickly and systems can be restored with minimal distribution. It is also crucial the health provider follow this to comply with GDPR to ensure they have the ability to restore patient data in a timely manner as they are the organisation accountable for the data.

5.Single SQL Database

Relying on a single SQL database is a critical point of failure that jeopardies both system availability and data integrity if the database was to failure due to the hardware all application services would become unavailable disrupting the patient care and all operations. **(Li and Manoharan, 2013)** Implementing and High Availability (HA) database removes the single point of failure it provides a automated failover system meaning upon failure the system automatically promotes a replica to become the new primary this minimises the outage time for the health provider, its data and services as well as this feature HA provides continuous backups and transaction logs dating exactly what has been carried out and when.

Bibliography

GDPR (2018). *Art. 5 GDPR – Principles Relating to Processing of Personal Data | General Data Protection Regulation (GDPR)*. [online] General Data Protection Regulation (GDPR). Available at: <https://gdpr-info.eu/art-5-gdpr/>.

ico.org.uk. (2023). *Fairness and transparency in data sharing*. [online] Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-sharing/data-sharing-a-code-of-practice/fairness-and-transparency-in-data-sharing/>.

(Cloudflare, 2025) <https://www.cloudflare.com/en-gb/learning/ssl/why-is-http-not-secure/>

Kamara, S., Fahmy, S., Schultz, E., Kerschbaum, F. and Fritzen, M. (2003). Analysis of vulnerabilities in Internet firewalls. *Computers & Security*, 22(3), pp.214–232. doi:[https://doi.org/10.1016/s0167-4048\(03\)00310-9](https://doi.org/10.1016/s0167-4048(03)00310-9).

Milos Ljubojevic, Bajic, A. and Danijel Mijic (2018). Centralized monitoring of computer networks using Zenoss open source platform. [online] 2, pp.1–5. doi:<https://doi.org/10.1109/infotech.2018.8345528>.

Alexander, D.E. (2015). *Disaster and Emergency Planning for Preparedness, Response, and Recovery*. [online] discovery.ucl.ac.uk. Available at: <https://discovery.ucl.ac.uk/id/eprint/1472594/>.

Li, Y. and Manoharan, S. (2013). A Performance Comparison of SQL and NoSQL Databases. *2013 IEEE Pacific Rim Conference on Communications, Computers and Signal Processing (PACRIM)*. [online] doi:<https://doi.org/10.1109/pacrim.2013.6625441>.

Lucca, G.A.D., Fasolino, A.R., Mastoianni, M. and Tramontana, P. (2004). *Identifying cross site scripting vulnerabilities in Web applications*. [online] IEEE Xplore. doi:<https://doi.org/10.1109/WSE.2004.10013>.

DO NOT COPY

Google Books. (2025). *SQL and Relational Theory*. [online] Available at:

https://books.google.co.uk/books?hl=en&lr=&id=WuZGD5tBfMwC&oi=fnd&pg=PR5&dq=what+is+sql+and+how+does+it+work+&ots=WisX_oIHLB&sig=hz84VlX0HQRpfzRqLwXK5HGqNxo&redir_esc=y#v=onepage&q=what%20is%20sql%20and%20how%20does%20it%20work&f=false [Accessed 13 Nov. 2025].

Ntantogian, C., Malliaros, S. and Xenakis, C. (2019). Evaluation of password hashing schemes in open source web platforms. *Computers & Security*, 84, pp.206–224.
doi:<https://doi.org/10.1016/j.cose.2019.03.011>.

